

GUION DOCENTE - RETRIEVAL AVANZADO

Desde Reranking hasta el pipeline completo

120 minutos | VS Code + Python + Terminal

OBJETIVO DE ESTE GUION

Documento preparado para leer casi literalmente durante la clase. En cada bloque se indica qué decir, qué mostrar en pantalla, cuándo ejecutar el código, qué preguntar al curso y qué tip de ingeniería de software reforzar.

REGLA DE RITMO

Antes de ejecutar: pide una predicción. Después de ejecutar: mira la evidencia. Luego cambia una sola variable y vuelve a probar.

Punto de partida: BLOQUE 5 - Reranking: retrieve wide, rerank narrow

Como usar este guion

LO QUE DICES

Texto pensado para leer casi literalmente. Puedes adaptarlo a tu tono, pero no necesitas improvisar la explicación técnica.

EN PANTALLA

Archivo, función, dibujo o fragmento que conviene tener visible mientras hablas.

EJECUTA

Comando que debes correr en ese momento. Antes de ejecutarlo, pregunta al curso qué creen que ocurrirá.

PREGUNTA AL CURSO

Pausa intencional. Espera una respuesta; no la contestes inmediatamente.

TIP DE INGENIERIA

Práctica de software, producción, observabilidad, performance o testing que vale la pena conectar con el código.

Mapa de 120 minutos

Minutos	Bloque	Resultado esperado
0-20	Repaso completo	Vector -> BM25 -> Hybrid -> RRF
20-45	Reranking	Entender CrossEncoder y 12_reranking.py
45-62	Query Rewriting	Entender por qué modificar la query
62-78	Multi-Query	Pasar de 1 consulta a N consultas
78-95	HyDE	Buscar usando un documento hipotético
95-115	Pipeline completo	Query Rewrite -> Hybrid -> RRF -> Reranker
115-120	Cierre	Reconstruir arquitectura sin

		mirar código
--	--	--------------

Antes de compartir pantalla

LO QUE DICES

Hoy no quiero perder tiempo instalando dependencias ni reconstruyendo el índice. Todo eso ya existe. Hoy nos vamos a concentrar en mejorar cómo seleccionamos la evidencia.

```
python --version
python -c "from pathlib import Path; print(Path.cwd())"
python -c "from sentence_transformers import CrossEncoder; CrossEncoder('cross-encoder/mmarco-mMiniLMv2-L12-H384-v1'); print('reranker OK')"
python -c "from pathlib import Path; p=Path('data/indice/vector_store.json'); print('indice:', p.exists(), p)"
```

TIP DE INGENIERIA

No muestres .env, API keys ni tokens. Si necesitas validar configuración, imprime solo nombres de modelos, rutas o booleanos.

Repaso de 20 minutos: donde estamos

DOCUMENTOS -> CHUNKS -> EMBEDDINGS -> INDEX -> RETRIEVAL -> CONTEXTO -> LLM -> RESPUESTA

LO QUE DICES

Antes de seguir quiero ubicar exactamente dónde estamos. En las clases anteriores tomamos documentos, los parseamos, los dividimos en chunks, generamos embeddings y construimos un índice. Hoy todavía no estamos trabajando en cómo responde el LLM. Estamos trabajando en una etapa anterior: decidir qué información merece llegar al LLM. Si recupero cinco documentos incorrectos y se los entrego a un modelo excelente, probablemente obtendré una respuesta muy bien escrita basada en evidencia incorrecta.

PREGUNTA AL CURSO

¿Cuál es el objetivo de Retrieval? Respuesta esperada: seleccionar la evidencia que queremos entregar al modelo generador.

1. Vector Search en 4 minutos

EN PANTALLA

Abre 08_consultar_indice.py y muestra solo el embedding de la query, la carga del store y store.search().

```
query_embedding = client.embeddings.create(
    model=settings.embedding_model,
    input=args.query,
).data[0].embedding

store = JsonVectorStore(INDEX_PATH)
store.load()
results = store.search(query_embedding, top_k=args.top_k)
```

LO QUE DICES

Voy a leer este código por responsabilidades. Primero convierto la pregunta a un vector. Segundo cargo los vectores que ya existen. Tercero comparo la pregunta con esos vectores y me quedo con los Top-K. Vector Search es especialmente útil cuando las palabras cambian, pero el significado se mantiene.

QUERY: "¿Cómo recupero acceso?"
DOCUMENTO: "Procedimiento de restablecimiento de credenciales"

Las palabras cambian, pero la intención puede ser similar.

PREGUNTA AL CURSO

Si el documento dice "restablecimiento de credenciales" y el usuario pregunta "cómo vuelvo a entrar a mi cuenta", ¿Vector Search podría relacionarlos? Respuesta esperada: sí.

2. BM25 en 4 minutos

VECTOR = SIGNIFICADO
BM25 = PALABRAS / TERMINOS EXACTOS

LO QUE DICES

Después agregamos una señal léxica. BM25 no intenta entender el significado profundo de la frase. Observa términos. Y eso es extremadamente útil cuando el identificador exacto importa.

F110-031
CVE-2026-12345
INC-13368
PROD-002524

LO QUE DICES

Si alguien busca F110-031, no quiero que el buscador piense demasiado. Quiero que encuentre F110-031. Esa es la fortaleza de una señal léxica.

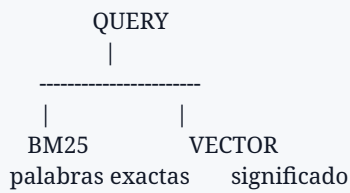
EJECUTA

```
python 09_busqueda_bm25.py --query "F110-031" --top-k 5
```

PREGUNTA AL CURSO

¿Qué ocurriría si en vez de un código exacto pregunto "cómo recuperar acceso a mi cuenta" y el documento dice "restablecer credenciales"? Respuesta: BM25 puede perder parte de esa relación semántica.

3. Hybrid Search en 4 minutos



LO QUE DICES

Hybrid Search significa correr al menos dos retrievers sobre el mismo corpus. Uno aporta coincidencia léxica; el otro similitud semántica. Todavía no fusionamos: primero observamos sus rankings por separado.

EJECUTA

```
python 10_busqueda_hibrida.py --query "Error F110-031 SAP" --top-k 5
```

PREGUNTA AL CURSO

BM25 me da 8.4 y Vector Search 0.83. ¿Puedo sumar $8.4 + 0.83$? Respuesta: no; son escalas diferentes.

4. RRF en 4 minutos



LO QUE DICES

RRF evita tratar de convertir BM25 y similitud coseno a una misma escala. En lugar de mirar el score original, mira la posición de cada documento en cada ranking. Si un documento aparece alto en varios retrievers, acumula apoyo.

$$\text{contribucion} = 1 / (k + \text{rank})$$

EJECUTA

```
python 11_rrf.py --query "Error F110-031 SAP" --candidate-k 10
```

LO QUE DICES

Miren $\text{ranks} = \{\text{bm25: 1, vector: 2}\}$. Esto es trazabilidad. No solo veo un score; puedo explicar cómo llegó un documento al ranking.

TRANSICION

Hasta aquí tenemos una buena máquina para encontrar candidatos. Ahora falta responder otra pregunta: de todos estos candidatos razonables, ¿cuáles son realmente los mejores?

Reranking: recuperar amplio, reordenar estrecho

QUERY -> BM25 + VECTOR -> RRF -> TOP 20 CANDIDATOS -> RERANKER -> TOP 5

LO QUE DICES

Hasta ahora BM25, Vector Search y RRF tienen una misión principal: encontrar candidatos. No necesitan ser perfectos. Necesitamos que el documento correcto quede dentro de ese conjunto. Ahora agregamos una segunda etapa cuya misión es ordenar esos candidatos con mayor precisión.

Retrieval: "no pierdas el documento correcto"

Reranking: "ahora ordénamelos mejor"

PREGUNTA AL CURSO

¿Por qué no usamos directamente el reranker contra millones de chunks? Respuesta esperada: porque evalúa pares query-documento, por lo que es mucho más costoso y lento.

Bi-encoder vs CrossEncoder

VECTOR SEARCH / BI-ENCODER

QUERY -----> encoder -----> vector Q
DOCUMENTO ----> encoder -----> vector D

$\text{cosine}(Q, D)$

CROSS-ENCODER

[QUERY + DOCUMENTO]

|
modelo

|
relevance score

LO QUE DICES

Con embeddings, query y documento se representan de manera independiente. Eso permite precomputar los documentos y buscar rápido. Un CrossEncoder recibe la query y el documento juntos. Puede evaluar directamente la relación entre ambos textos. Normalmente ganamos precisión, pero pagamos más latencia y costo computacional.

TIP DE INGENIERIA

Esta separación de responsabilidades es muy común en sistemas de búsqueda: primera etapa orientada a recall y velocidad; segunda etapa orientada a precisión.

5.1 rerank() - lectura línea por línea

EN PANTALLA

Abre `retrieval_advanced.py` y baja directamente a `def rerank(...)`. No muestres todavía `12_reranking.py`.

```
def rerank(query: str, candidates: list, top_k: int = 5):
    from sentence_transformers import CrossEncoder

    model = CrossEncoder(
        "cross-encoder/mmarco-mMiniLMv2-L12-H384-v1"
    )

    pairs = [
        (query, item["chunk"].text)
        for item in candidates
    ]

    scores = model.predict(pairs)

    reranked = []
    for item, score in zip(candidates, scores):
        enriched = dict(item)
        enriched["rerank_score"] = float(score)
        reranked.append(enriched)

    reranked.sort(
        key=lambda item: item["rerank_score"],
        reverse=True,
    )

    return reranked[:top_k]
```

Firma de la función

```
def rerank(query: str, candidates: list, top_k: int = 5):
```

LO QUE DICES

La función recibe tres cosas. Uno: la pregunta. Dos: una lista de candidatos que ya sobrevivieron al retrieval anterior. Tres: cuántos resultados quiero devolver. `Candidates` no representa toda la colección; representa un subconjunto pequeño.

PREGUNTA AL CURSO

Si `candidate_k` es 20 y `top_k` es 5, ¿cuántos documentos ve el reranker y cuántos devuelve? Respuesta: evalúa hasta 20 y devuelve 5.

Carga del CrossEncoder

```
from sentence_transformers import CrossEncoder

model = CrossEncoder(
    "cross-encoder/mmarco-mMiniLMv2-L12-H384-v1"
)
```

LO QUE DICES

Estoy usando sentence-transformers, pero aquí no estoy creando embeddings tradicionales. Estoy cargando un CrossEncoder: un modelo que puntúa directamente la relación entre una query y un documento.

TIP DE INGENIERIA

El código educativo carga el modelo dentro de la función para que sea autocontenido. En producción no conviene cargar un modelo pesado en cada request. Se inicializa una vez al levantar el servicio y se reutiliza, o se inyecta como dependencia.

```
# Idea productiva
reranker_model = CrossEncoder(MODEL_NAME)

def rerank(query, candidates, model=reranker_model):
    ...
```

La línea más importante: pairs

```
pairs = [
    (query, item["chunk"].text)
    for item in candidates
]
```

LO QUE DICES

Voy a traducir este Python a castellano. Quiero construir una lista. Recorro cada candidato y, por cada uno, creo una tupla con dos elementos: la misma query y el texto de ese chunk. La pregunta se repite; lo que cambia es el documento.

```
(
    "¿Por qué falló F110-031?",
    "El error F110-031 aparece durante el proceso automático..."
)

(
    "¿Por qué falló F110-031?",
    "SAP puede producir errores administrativos por múltiples causas..."
)
```

PREGUNTA AL CURSO

¿Qué diferencia hay con Vector Search? Respuesta: antes query y documento se codificaban independientemente; aquí se evalúan juntos como un par.

model.predict(pairs)

```
scores = model.predict(pairs)
```

LO QUE DICES

Le paso todos los pares al modelo. El modelo devuelve un score de relevancia para cada par. No lo interpreten automáticamente como probabilidad. En esta clase nos interesa principalmente el orden relativo entre candidatos.

```
pair 1 -> 7.12  
pair 2 -> 3.83  
pair 3 -> -1.42
```

TIP DE INGENIERIA

No hardcodees umbrales sobre un score de modelo sin calibrarlo. Primero evalúa distribución, estabilidad por dominio y relación con relevancia real.

zip(candidates, scores)

```
for item, score in zip(candidates, scores):
```

LO QUE DICES

zip pega la primera posición de candidates con la primera de scores, la segunda con la segunda y así sucesivamente. Es la forma de volver a unir cada candidato con su puntuación.

```
candidates = ["A", "B", "C"]  
scores     = [8.3, 2.1, 6.4]  
  
zip -> (A, 8.3), (B, 2.1), (C, 6.4)
```

Enriquecer sin perder trazabilidad

```
enriched = dict(item)  
enriched["rerank_score"] = float(score)  
reranked.append(enriched)
```

LO QUE DICES

Creamos una copia del item original en vez de destruirlo. Conservamos chunk, rrf_score y ranks, y agregamos rerank_score. Así mantenemos la historia de cómo llegó cada documento hasta la salida.

```
{
  "chunk": ...,
  "rrf_score": 0.032,
  "ranks": {"bm25": 1, "vector": 3},
  "rerank_score": 7.14
}
```

TIP DE INGENIERIA

Observabilidad: no guardes solo el resultado final. Guarda las señales intermedias suficientes para diagnosticar por qué un documento entró, subió, bajó o desapareció.

Ordenar y cortar Top-K

```
reranked.sort(
    key=lambda item: item["rerank_score"],
    reverse=True,
)

return reranked[:top_k]
```

LO QUE DICES

sort ordena la lista usando rerank_score. reverse=True significa de mayor a menor. Después usamos slicing: si tengo veinte candidatos y top_k es cinco, devuelvo las primeras cinco posiciones.

PREGUNTA AL CURSO

¿Qué pasaría con reverse=False? Respuesta: pondríamos primero los scores menores, es decir, invertiríamos el ranking.

5.2 12_reranking.py - leer la arquitectura, no repetir funciones

```
bm25 = bm25_search(args.query, top_k=args.candidate_k)
vector = vector_search(client, settings, args.query, top_k=args.candidate_k)
fused = reciprocal_rank_fusion({"bm25": bm25, "vector": vector})
final = rerank(args.query, fused[:args.candidate_k], top_k=args.top_k)
```

LO QUE DICES

Lean estas cuatro líneas como arquitectura: primera opinión, BM25. Segunda opinión, Vector Search. Fusiono rankings con RRF. Y recién después uso el modelo más caro para reordenar el conjunto pequeño.

EJECUTA 1

```
python 11_rrf.py --query "Error F110-031 SAP" --candidate-k 10
```

LO QUE DICES

Antes del reranker quiero que miren el orden RRF. Memorícenlo visualmente: veremos si cambia.

EJECUTA 2

```
python 12_reranking.py --query "Error F110-031 SAP" --candidate-k 20 --top-k 5
```

LO QUE DICES

Ahora tenemos dos capas de score visibles. rrf_score explica cómo llegó el candidato. rerank_score explica cómo quedó ordenado al final. Esto es mucho mejor que tratar Retrieval como una caja negra.

CIERRE DEL BLOQUE

Retrieve wide, rerank narrow. Recuperamos con amplitud; reordenamos con precisión.

Query Rewriting: cuando el problema está en la pregunta

Usuario: "¿por qué falló lo de ayer?"

LO QUE DICES

Hasta ahora asumimos algo peligroso: que la query del usuario estaba bien formulada. En sistemas conversacionales eso muchas veces es falso. ¿Qué sistema? ¿Qué proceso? ¿Qué error? ¿Qué significa ayer? El problema puede no estar en los documentos; puede estar en la consulta.

PREGUNTA AL CURSO

¿Qué términos buscables faltan en "por qué falló lo de ayer"? Esperado: sistema, proceso, código, fecha y contexto conversacional.

HISTORIAL

"Estamos revisando SAP F110. Apareció F110-031."

QUERY ORIGINAL

"¿Por qué falló lo de ayer?"

QUERY REWRITTEN

"causa error F110-031 proceso pagos SAP F110 2026-09-03"

LO QUE DICES

El LLM aquí NO responde la pregunta. Su trabajo es transformar una expresión humana, ambigua y conversacional en una consulta más buscable.

6.1 rewrite_query()

```
def rewrite_query(client, settings, query: str, history: str = "") -> str:
    prompt = f"""
    Convierte la consulta del usuario en una consulta optimizada
    para recuperacion documental.

    Reglas:
    - conserva nombres, fechas, codigos, IDs y terminos tecnicos;
    - usa el historial solo para resolver referencias ambiguas;
    - no respondas la pregunta;
    - devuelve solamente la consulta optimizada.

    Historial:
    {history}

    Consulta original:
    {query}
    """.strip()

    response = client.responses.create(
        model=generation_model(settings),
        input=prompt,
    )

    return response.output_text.strip()
```

LO QUE DICES

history tiene valor por defecto vacío. Eso hace que la función también funcione sin conversación previa. El prompt contiene cuatro reglas. Las dos más importantes para hoy son: no respondas la pregunta y conserva códigos, IDs, nombres y fechas.

TIP DE INGENIERIA

Si un rewriter transforma F110-031 en una descripción genérica, destruye una señal exacta que BM25 estaba diseñado para proteger. Las transformaciones de query también necesitan pruebas y guardrails.

LO QUE DICES

client.responses.create hace una llamada generativa, pero el resultado no se muestra al usuario. Vuelve al pipeline de retrieval. output_text.strip() toma el texto generado y elimina espacios o saltos innecesarios en los extremos.

6.2 13_query_rewriting.py

```
query = "¿Por que fallo lo de ayer?"
history = (
    "Estamos revisando el proceso automatico de pagos SAP F110 "
    "ejecutado el 2026-09-03. Aparecio el codigo F110-031."
)

rewritten = rewrite_query(client, settings, query, history)
print("ORIGINAL :", query)
print("REWRITTEN:", rewritten)
```

PREGUNTA AL CURSO

Antes de ejecutar: ¿qué consulta creen que debería producir el sistema? Pide una respuesta concreta de uno o dos alumnos.

EJECUTA

```
python 13_query_rewriting.py
```

LO QUE DICES

No evalúen si la nueva frase suena bonita. Evalúen si es más buscable. Query Rewriting se mide por el retrieval que produce: recall, relevancia y posición del documento correcto.

TIP DE INGENIERIA

En producción registra al menos original_query, rewritten_query, timestamp y resultados de retrieval. Si empeora el sistema, necesitas distinguir si falló el retriever o el rewriter.

Multi-Query: una pregunta, varias rutas de búsqueda

QUERY REWRITING: 1 -> 1
MULTI-QUERY: 1 -> N

LO QUE DICES

Query Rewriting busca una mejor formulación de la misma consulta. Multi-Query hace otra cosa: genera varias consultas que cubren ángulos distintos del problema. El objetivo es aumentar cobertura.

Pregunta original:
"¿Cómo mejorar el rendimiento de mi pipeline RAG?"

Posibles rutas:

1. optimizar retrieval y ranking en RAG
2. reducir latencia de embeddings y búsqueda
3. mejorar chunking, recall y reranking

CASO REAL

En investigación de incidentes, una misma pregunta puede abrir una ruta por logs, otra por infraestructura y otra por cambios de aplicación. Multi-Query formaliza esa idea.

7.1 generate_queries()

```
def generate_queries(client, settings, query: str, n: int = 3) -> list[str]:
    prompt = f"""
    Genera {n} consultas de búsqueda diferentes para recuperar
    informacion que ayude a responder la pregunta original.
    Cada consulta debe cubrir un angulo distinto.
    No respondas la pregunta.
    Devuelve una consulta por linea y nada mas.
    Pregunta:
    {query}
    """.strip()

    response = client.responses.create(
        model=generation_model(settings),
        input=prompt,
    )

    queries = []
    for line in response.output_text.splitlines():
        line = re.sub(r"^\s*[-(d\.)]+\s*", "", line).strip()
        if line:
            queries.append(line)

    return queries[:n]
```

LO QUE DICES

n es el máximo de consultas que quiero. El prompt pide una consulta por línea para que después sea fácil procesar la respuesta. Aun así, no confío ciegamente en que el LLM respete el formato.

splitlines() y limpieza

```
for line in response.output_text.splitlines():
```

LO QUE DICES

splitlines convierte el texto completo en líneas individuales. Si el modelo devuelve tres líneas, puedo procesarlas una por una.

ANTES

1. mejorar embeddings
- mejorar retrieval
- 2) optimizar chunking

DESPUES DE LIMPIAR

- mejorar embeddings
- mejorar retrieval
- optimizar chunking

```
line = re.sub(r"^\s*[^\d\.\,)]+\s*", "", line).strip()
```

LO QUE DICES

La expresión regular limpia numeración, guiones, puntos o paréntesis al principio. No necesito enseñar regex en profundidad; lo importante es mostrar que el software normaliza la salida del LLM antes de usarla.

TIP DE INGENIERIA

Una instrucción en el prompt no es una garantía. Si el formato es importante para el sistema, valida también en código. En producción, Structured Outputs suele ser mejor que parsear texto libre.

```
if line:
    queries.append(line)

return queries[:n]
```

LO QUE DICES

Solo guardo líneas no vacías. Y aunque el modelo devuelva más consultas de las pedidas, mi función aplica el contrato y corta a n.

7.2 Ejecutar Multi-Query

PREGUNTA AL CURSO

Antes de ejecutar: ¿qué tres ángulos distintos esperan que genere para "¿Cómo mejorar el rendimiento de mi pipeline RAG?"

EJECUTA

```
python 14_multi_query.py
```

LO QUE DICES

La gracia no es que tres queries siempre sean mejores. La gracia es cobertura. Pero cada query adicional también puede agregar embeddings, búsquedas, latencia y complejidad de fusión.

PREGUNTA AL CURSO

¿Usarían Multi-Query por defecto en todas las consultas? Respuesta esperada: no; primero debe demostrar mejora medible.

HyDE: buscar con un documento hipotético

LO QUE DICES

Ahora viene la técnica más contraintuitiva. Una pregunta suele ser corta y abstracta. Un manual técnico suele ser largo, explicativo y concreto. HyDE intenta cerrar esa brecha generando un documento hipotético parecido al tipo de documento que esperamos encontrar.

QUERY

"¿Por qué puede fallar un proceso automático de pagos?"

↓ LLM

DOCUMENTO HIPOTETICO

"Los procesos automáticos de pago pueden fallar por configuración de sociedad, métodos de pago, autorizaciones o datos maestros..."

↓ embedding

VECTOR SEARCH

↓

DOCUMENTOS REALES

PREGUNTA AL CURSO

¿El documento hipotético tiene que ser una fuente verdadera? Respuesta: no. No se usa como evidencia final; solo como puente para buscar.

8.1 generate_hypothetical_document()

```
def generate_hypothetical_document(client, settings, query: str) -> str:
    prompt = f"""
    Escribe un breve documento hipotetico que podria contener
    informacion relevante para responder la consulta.
    No cites fuentes.
    No presentes el texto al usuario como respuesta.
    El objetivo es producir un texto parecido al tipo de documento
    que esperamos recuperar.
    Consulta:
    {query}
    """.strip()

    response = client.responses.create(
        model=generation_model(settings),
        input=prompt,
    )

    return response.output_text.strip()
```

LO QUE DICES

El prompt deja una frontera muy clara: no cites fuentes y no presentes este texto al usuario como respuesta. El objetivo no es responder; es generar una representación textual que ayude a la búsqueda.

8.2 hyde_search()

```
def hyde_search(client, settings, query: str, top_k: int = 5):  
    hypothetical = generate_hypothetical_document(  
        client, settings, query  
    )  
  
    results = vector_search(  
        client, settings, hypothetical, top_k=top_k  
    )  
  
    return hypothetical, results
```

LO QUE DICES

Lean la función como tres pasos. Uno: genero un documento hipotético. Dos: reutilizo `vector_search`. Tres: atención, a `vector_search` no le paso la query original; le paso `hypothetical`. Esa línea contiene la esencia de HyDE.

TIP DE INGENIERIA

HyDE puede ayudar, pero también puede desviar la búsqueda si el documento hipotético inventa una dirección incorrecta. Por eso se evalúa con métricas, no por sofisticación.

8.3 Ejecutar HyDE

PREGUNTA AL CURSO

Antes de ejecutar: ¿qué tipo de texto esperan que genere el documento hipotético? ¿Un resumen, una lista, una respuesta, un manual?

EJECUTA

```
python 15_hyde.py --query "¿Por que puede fallar un proceso automatico de pagos?"
```

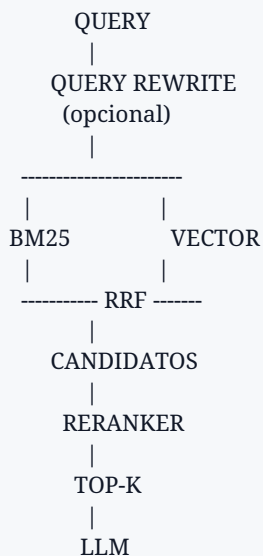
LO QUE DICES

Primero miren el documento hipotético. Después miren exclusivamente los documentos reales recuperados. La evidencia válida sigue siendo la colección real. HyDE solo modificó el objeto que llevamos al espacio de embeddings.

CIERRE DEL BLOQUE

Query Rewriting cambia la formulación. Multi-Query cambia la cobertura. HyDE cambia el objeto que embeddeamos.

Integrar el pipeline moderno



LO QUE DICES

Ahora quiero que dejemos de pensar en técnicas aisladas. Pensemos en responsabilidades. Una base muy fuerte para empezar es Query Rewriting opcional, Hybrid Search, RRF y Reranking. Multi-Query y HyDE se activan cuando el tipo de consulta y las métricas justifican su costo.

9.1 advanced_retrieval()

```
def advanced_retrieval(
    client,
    settings,
    query: str,
    history: str = "",
    candidate_k: int = 20,
    top_k: int = 5,
    use_rewrite: bool = True,
):
```

LO QUE DICES

Aquí aparecen dos K. `candidate_k` controla cuántos candidatos dejamos entrar a la etapa de precisión. `top_k` controla cuántos documentos entregamos finalmente. Como intuición: `candidate_k` favorece cobertura; `top_k` controla el contexto final.

PREGUNTA AL CURSO

¿Por qué no usar candidate_k=5 y top_k=5 siempre? Respuesta: porque si la primera etapa corta demasiado pronto, el reranker nunca verá documentos relevantes que quedaron fuera.

Paso 1 - Elegir la query de búsqueda

```
search_query = query

if use_rewrite:
    search_query = rewrite_query(
        client, settings, query, history
    )
```

LO QUE DICES

Por defecto empiezo con la query original. Si use_rewrite está activo, la reemplazo por una versión optimizada. Esta decisión es configurable; no quiero quedar amarrado a usar rewriting en todas las consultas.

TIP DE INGENIERIA

Un flag como use_rewrite permite feature flags, pruebas A/B y rollback. Las técnicas de IA también deben poder activarse y desactivarse operativamente.

Paso 2 - Recuperar por dos señales

```
bm25 = bm25_search(
    search_query,
    top_k=candidate_k
)

vector = vector_search(
    client,
    settings,
    search_query,
    top_k=candidate_k
)
```

LO QUE DICES

Primera señal: coincidencia exacta. Segunda señal: similitud semántica. Ambos recuperan sobre el mismo corpus, pero con funciones de relevancia distintas.

Paso 3 - Fusionar

```
fused = reciprocal_rank_fusion({
    "bm25": bm25,
    "vector": vector,
})
```

LO QUE DICES

Ahora fusionamos rankings sin tratar de sumar scores incompatibles. RRF nos entrega un conjunto ordenado y con trazabilidad de posiciones.

Paso 4 - Rerankear

```
final = rerank(
    search_query,
    fused[:candidate_k],
    top_k=top_k,
)

return search_query, final
```

LO QUE DICES

Tomo el conjunto fusionado, lo limito a candidate_k, lo paso al CrossEncoder y devuelvo top_k. Esta es la idea completa: recuperar amplio con métodos rápidos; reordenar estrecho con un método más costoso.

Pausa de ingeniería de software: esto ya es un sistema

```
bm25_search()
vector_search()
reciprocal_rank_fusion()
rerank()
rewrite_query()
advanced_retrieval()
```

LO QUE DICES

Fíjense que esto ya parece software y no solamente una demo de IA. Cada función tiene una responsabilidad. Eso permite probar, medir y reemplazar componentes individualmente. Cuando el sistema falla, necesito aislar qué componente falló.

- Separación de responsabilidades: cada retriever se prueba por separado.
- Configuración explícita: modelos, candidate_k, top_k y flags fuera de la lógica principal.
- Observabilidad: registrar query original, query buscable, rankings, scores y latencias.
- Testing: dataset de queries con documentos relevantes esperados y métricas como Recall@K, MRR y nDCG.
- Performance: medir latencia por etapa, no solo tiempo total.
- Costo: activar técnicas solo si aportan mejora medible.

9.2 Ejecución final

EJECUTA 1

```
python 16_retrieval_avanzado.py --query "Error F110-031 SAP" --candidate-k 20 --top-k 5
```

PREGUNTA AL CURSO

¿Qué componentes participaron hasta llegar a la salida? Esperado: Query Rewrite si está activo, BM25, Vector, RRF y Reranker.

EJECUTA 2

```
python 16_retrieval_avanzado.py --query "¿Por que fallo lo de ayer?" --history "Revisamos SAP F110 del 2026-09-03 y aparecio F110-031" --top-k 5
```

LO QUE DICES

Miren QUERY ORIGINAL y QUERY BUSQUEDA. Esta es una diferencia muy importante en sistemas conversacionales: una cosa es lo que escribió el humano y otra puede ser la representación que usamos para recuperar evidencia.

Caso real para contar en clase

LO QUE DICES

Supongamos que esto es un agente de soporte industrial. El operador escribe: "me apareció otra vez el error del pago". La conversación anterior contenía F110-031. Query Rewriting resuelve la referencia. BM25 protege F110-031. Vector Search recupera documentos conceptualmente relacionados. RRF combina ambas señales. Reranking mira cada candidato con mayor precisión. Y recién después entregamos los mejores chunks al LLM.

PREGUNTA AL CURSO

¿Cuál de todas esas piezas es el LLM que responde finalmente al usuario? Respuesta: ninguna todavía. El generador viene después del Top-K final.

Cierre sin mirar el código

LO QUE DICES

Voy a borrar la arquitectura. Ustedes me la van a reconstruir solo por responsabilidades.

USUARIO

|

QUERY

|

???

PREGUNTA AL CURSO

¿Qué hago si la query depende de la conversación?

Respuesta esperada: Query Rewriting.

PREGUNTA AL CURSO

¿Qué recupera identificadores, códigos y palabras exactas?

Respuesta esperada: BM25.

PREGUNTA AL CURSO

¿Qué recupera significado, sinónimos y paráfrasis?

Respuesta esperada: Vector Search.

PREGUNTA AL CURSO

¿Cómo junto rankings con escalas distintas?

Respuesta esperada: RRF.

PREGUNTA AL CURSO

¿Qué hago con mis candidatos antes de entregar el Top-K?

Respuesta esperada: Reranking.

PREGUNTA AL CURSO

¿Qué hace Multi-Query?

Respuesta esperada: Convierte una consulta en N rutas para aumentar cobertura.

PREGUNTA AL CURSO

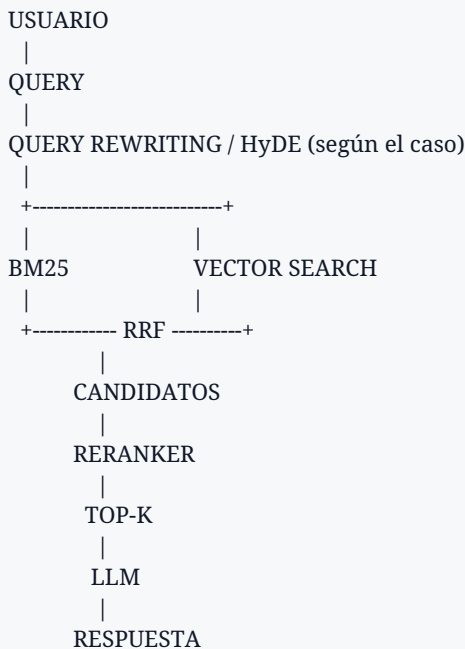
¿Qué hace HyDE?

Respuesta esperada: Genera un documento hipotético para buscar; no lo usa como evidencia.

PREGUNTA AL CURSO

¿Qué recibe finalmente el LLM generador?

Respuesta esperada: Los Top-K chunks reales seleccionados.



FRASE FINAL

Hoy no hicimos al LLM más inteligente. Hicimos al sistema mejor seleccionando evidencia. Esa diferencia es importante: AI Engineering no es solamente elegir un modelo; es diseñar, instrumentar, probar y medir el pipeline que lo rodea.

5 ideas de ingeniería que quiero que recuerden

- 1. No cargues modelos pesados por request.** Inicializa modelos al levantar el servicio y reutilízalos.
- 2. No pierdas observabilidad.** Conserva query original, query transformada, rankings, scores, latencias y fuentes.
- 3. Mantén componentes separados.** BM25, Vector, RRF, Reranker y Rewriter deben poder probarse individualmente.
- 4. Más técnicas no significa mejor arquitectura.** Agrega complejidad solo cuando una métrica demuestra que la necesitas.

5. Evalúa retrieval antes del LLM. Mide Recall@K, MRR, nDCG, Hit Rate@K, latencia y costo; no evalúes el RAG solo porque la respuesta "suena bien".

SI VAS ATRASADO

No sacrifiques la integración Hybrid + RRF + Reranking. Puedes explicar Multi-Query o HyDE sin ejecutar, pero termina mostrando el pipeline completo y dónde entra el LLM generador.

Chuleta de comandos para la clase

```
# Repaso
python 09_busqueda_bm25.py --query "F110-031" --top-k 5
python 10_busqueda_hibrida.py --query "Error F110-031 SAP" --top-k 5
python 11_rrf.py --query "Error F110-031 SAP" --candidate-k 10

# Reranking
python 12_reranking.py --query "Error F110-031 SAP" --candidate-k 20 --top-k 5

# Query Rewriting
python 13_query_rewriting.py

# Multi-Query
python 14_multi_query.py

# HyDE
python 15_hyde.py --query "¿Por que puede fallar un proceso automatico de pagos?"

# Pipeline final
python 16_retrieval_avanzado.py --query "Error F110-031 SAP" --candidate-k 20 --top-k 5
python 16_retrieval_avanzado.py --query "¿Por que fallo lo de ayer?" --history "Revisamos SAP F110 del 2026-09-03 y aparecio F110-031" --top-k 5
```